

## UNIT 3

### INHERITANCE

The mechanism of deriving a new class from an old one is called **inheritance** or **derivation**. The old class is referred to as the **base class** and the new one is called the **derived class** or **sub class**. The derived class inherits some or all of the traits from the base class. A class can also inherit properties from more than one class or from more than one level. Reusability is an important feature of OOP. A derived class can be defined by specifying its relationship with the base class in addition to its own details.

class derived-class-name :

```
    visibility-mode base-class-name{  
        .....  
        .....  
    }
```

The **colon** indicates that the derived class name is derived from the base-class-name. the visibility mode is optional and if present, may be either **private or protected or public**. The default is private. Visibility mode specifies whether the features of the base class are privately derived or publicly derived.

class ABC : private XYZ

```
{  
    members of ABC;  
};
```

//private derivation

class ABC :public XYZ

```
{  
    members of ABC;  
};
```

```
//public derivation
class ABC:protected XYZ {
// protected derivationmembers of ABC;
};
class ABC:XYZ //private by default
{
members of ABC;
};
```

When a base class is **privately inherited** by a derived class, public members of the base class can only be accessed by the member functions of the derived class. Private members of base class are inaccessible to the objects of the derived class

When a base class is **protected inherited** by a derived class, public members of the base class can only be accessed by the member functions of the derived class. Private members of base class are inaccessible to the objects of the derived class. If private members of base class are to be inherited to derived class then declare them as protected. When the base class is **publicly inherited**, public members of the base class is publicly inherited, public members of the base class become public members of the derived class and therefore they are accessible to the objects of the derived class. In both the cases, the private members are not inherited and therefore, the private members of a base class will never become the members of its derived class.

In inheritance, some of the base class data elements and member functions are inherited into the derived class.

We can add our own data and member functions and thus extend the functionality of the base class.

Inheritance, when used to modify and extend the capability of the existing classes, becomes a very powerful tool for incremental program development.

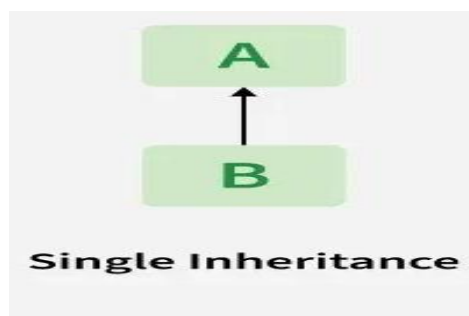
## Types of Inheritance:

1. Single Inheritance
2. Multi level Inheritance
3. Multiple Inheritance
4. Hybrid inheritance
5. Hierarchical Inheritance

|            | Derived Class | Derived Class  | Derived Class |
|------------|---------------|----------------|---------------|
| Base Class | Private Mode  | Protected Mode | Public Mode   |
| Private    | Not Inherited | Not Inherited  | Not Inherited |
| Protected  | Private       | Protected      | Protected     |
| Public     | Private       | Protected      | Public        |

### 1. SINGLE INHERITANCE:

One derived class inherits from only one base class. It is the most simplest form of Inheritance.



```
#include<iostream>

using namespace std;

class A
{
public:
```

```

int a,b;

void get()
{ cout<<"Enter any two Integer values"<<endl;
  cin>>a>>b;
}

};

class B:public A
{ int c;
  public:
  void add()
  {
    c=a+b;
    cout<<a<<"+"<<b<<"="<<c;
  }
};

int main()
{
  B b;
  b.get();
  b.add();
}

```

Output:

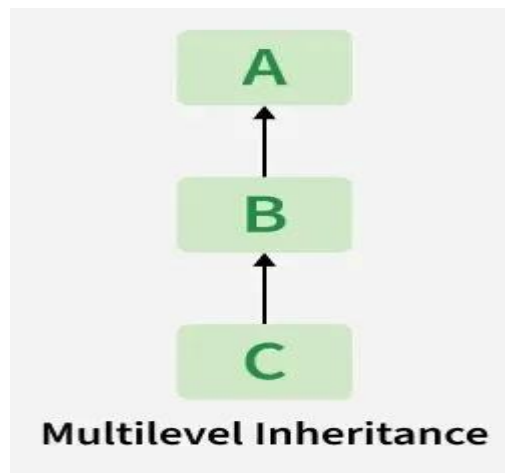
Enter any two Integer values

1 2

1+2 =3

## **2. MULTILEVEL INHERITANCE:**

In this type of inheritance the derived class inherits from a class, which in turn inherits from some other class. The Super class for one, is sub class for the other.



```
#include<iostream.h>

class A
{
public:
int a,b;
void get()
{
    cout<<"Enter any two Integer values"<<endl;
    cin>>a>>b;
}
};

class B:public A
{
public:
int c;
```

```

void add()
{
    c=a+b;
}
};

class C:public B
{
    public:
        void show()
        {
            cout<<a<<"+"<<b<<"="<<c;
        }
};

int main()
{
    C c;
    c.get();
    c.add();
    c.show();
}

```

Output:

Enter any two Integer values

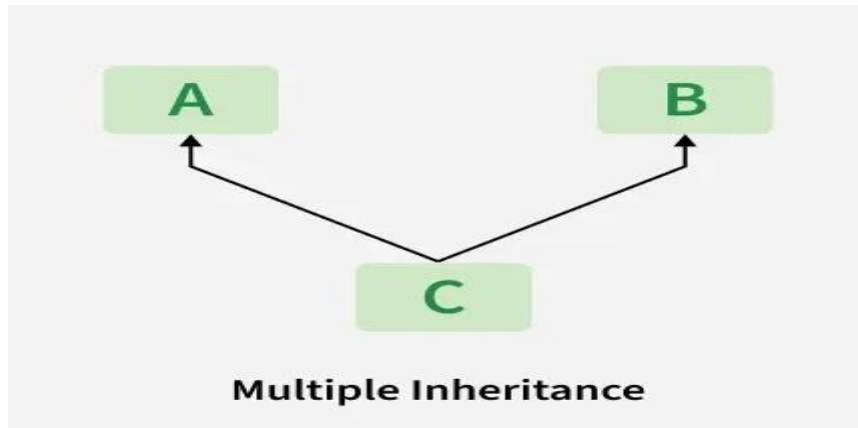
12

14

12+14=26

### 3. MULTIPLE INHERITANCE:

In this type of inheritance a single derived class may inherit from two or more than two base classes.



Syntax:

```
class D : visibility A, visibility B,....
```

```
{
```

```
.....
```

```
}
```

```
#include<iostream.h>
```

```
class A
```

```
{ public:
```

```
    int a;
```

```
    void getA()
```

```
    { cout<<"Enter an Integer value"<<endl;
```

```
      cin>>a;
```

```
    }
```

```
};
```

```
class B
```

```
{ public:
```

```

int b;

void getB()
    { cout<<"Enter an Integer value"<<endl;
      cin>>b;
    }

};

class C:public A,public B
{ public:
    int c;

    void add()
    {
        c=a+b;

        cout<<a<<"+"<<b<<"="<<c<<endl;
    }

};

int main()
{ C obj;

  obj.getA();

  obj.getB();

  obj.add();

}

```

### **Output:**

Enter an Integer

value 12 Enter anInteger value 13

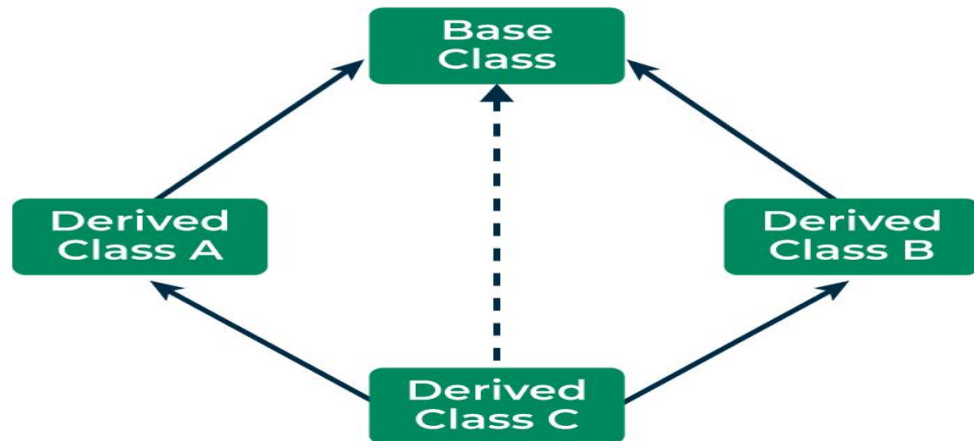
12+13=25



#### 4. HYBRID INHERITANCE:

Hybrid inheritance is combination of two or more inheritances such as single,multiple,multilevel or Hierarchical inheritances.

#### Hybrid Inheritance



```
#include<iostream.h>

class arithmetic
{ protected:
    int num1, num2;
    public:
    void getdata()
    {
        cout<<"For Addition:";
        cout<<"\nEnter the first number: ";
        cin>>num1;
        cout<<"\nEnter the second number: ";
        cin>>num2;
    }
};

class plus:public arithmetic
```

```

{
    protected:
    int sum;
    public:
    void add()
    {
        sum=num1+num2;
    }
};

class minus
{
    protected:
    int n1,n2,diff;
    public:
    void sub()
    {
        cout<<"\nFor Subtraction:";
        cout<<"\nEnter the first number: ";
        cin>>n1;
        cout<<"\nEnter the second number: ";
        cin>>n2;
        diff=n1-n2;
    }
};

class result:public plus, public minus

```

```

{
    public:
        void display()
        {
            cout<<"\nSum of "<<num1<<" and "<<num2<<"= "<<sum;
            cout<<"\nDifference of "<<n1<<" and "<<n2<<"= "<<diff;
        }
};

int main()
{
    result z;
    z.getdata();
    z.add();
    z.sub();
    z.display();
}

```

### **Output:**

For Addition:

Enter the first number: 1

Enter the second number: 2

For Subtraction:

Enter the first number: 3

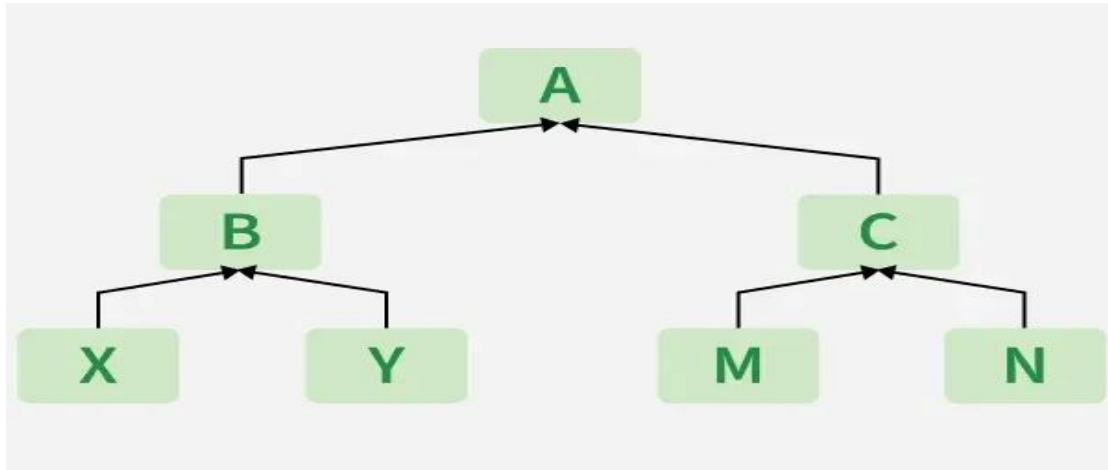
Enter the second number: 4

Sum of 1 and 2= 3

Difference of 3 and 4= -1

## 5. HIERARCHICAL INHERIT:-

Inheriting is a method of inheritance where one or more derived classes is derived from common base class.



```
#include<iostream.h>

class A //Base Class
{
public:
    int a,b;
    void getnumber()
    {
        cout<<"\n\nEnter Number :\t";
        cin>>a;
    }
};

class B : public A //Derived Class 1
{
public:
    void square()
```

```

    {
    getnumber(); //Call Base class property
    cout<<"\n\n\tSquare of the number :\t"<<(a*a);}

    };

class C :public A //Derived Class 2
{
public:
void cube()
{
    getnumber(); //Call Base class property
    cout<<"\n\n\tCube of the number ::: \t"<<(a*a*a);
}

};

int main()
{
    B b1; //b1 is object of Derived class 1
    b1.square(); //call member function of class B
    C c1; //c1 is object of Derived class 2
    c1.cube(); //call member function of class C
}

```

### **Output:**

Enter Number : 2

Square of the number : 4

Enter Number : 3

Cube of the number ::: 27

# Function Overriding in C++

A function is a block of code that performs a specific task. It takes input, processes it, and returns an output. Function Overriding in C++ is a type of polymorphism where a derived class redefines a function from its base class using the same name, return type, and parameters (i.e., the same function signature).

## Conditions:

- The base class function must be virtual.
- The derived class must use the same signature.
- It's a form of runtime polymorphism (dynamic binding).

## How Does It Work?

- If you call the function using a **base class object**, the **base class version** is executed.
- If you call it using a **derived class object**, the **derived class version** is executed.

## Example:

```
class Animal {
public:
    virtual void sound() {
        cout << "Animal makes a sound\n";
    }
};

class Dog : public Animal {
public:
    void sound() override { // Correct override
        cout << "Dog barks\n";
    }
};
```

## Override Keyword in C++

To avoid mistakes when overriding functions, C++ introduced the override keyword.

### Why use override?

- Ensures you're actually overriding a virtual function.
- If the signature doesn't match any base class function, the compiler gives an error.

### Mistake Caught Example:

```
class Dog : public Animal {  
  
public:  
    void Sound() override { // Error: No matching function in base class  
        cout << "Dog barks\n";  
    }  
};
```

Without override, this would compile but not override the base function - leading to unexpected behaviour.

### Advantages of Function Overriding

1. Dynamic behavior at runtime.
2. Promotes code reuse through inheritance.
3. Changes in base class can affect derived classes easily.
4. Enables design patterns like Strategy, Command, etc.
5. Encourages loose coupling by coding to base class interfaces.

### Limitations of Function Overriding

1. Slightly slower than normal function calls due to virtual table lookup.
2. Adds memory overhead (vtable, vptr).
3. Complexity increases in large inheritance hierarchies.
4. Mistakes in overriding may cause bugs if signatures don't match.
5. Requires careful design to avoid maintenance issues.

# Pure Virtual Functions and Abstract Classes in C++

A pure virtual function is a virtual function with no implementation in the base class, declared using `= 0`. A class with at least one pure virtual function is an abstract class that cannot be instantiated and serves as a blueprint for derived classes, which must provide their own implementation.

- ◆ A class with at least one pure virtual function becomes an abstract class and Objects of abstract classes cannot be created directly.
- ◆ Abstract classes are used to define interfaces and ensure common structure among derived classes.
- ◆ Useful in polymorphism where different classes share the same interface but have different behaviors.
- ◆ A pure virtual function forces derived classes to override it.
- ◆ `virtual void draw() = 0;` declares a pure virtual function, forcing derived classes to provide their own implementation.

```
#include <iostream>
using namespace std;
```

```
class Shape {
public:
    virtual void draw() = 0; // Pure virtual function
};
```

```
class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle\n";
    }
};
```

```
int main() {

    // Error in the below line : Cannot create
    // object of abstract class
    // Shape s;

    // Pointer to abstract class
    Shape* s = new Circle();

    // Output: Drawing Circle
```



```
s->draw();  
  
delete s;  
}
```

**Output**

Drawing Circle

**Explanation:** This example shows an abstract class Shape with a pure virtual function draw(). Circle overrides draw(), and using a Shape pointer to a Circle demonstrates polymorphism, outputting Drawing Circle.

## **Pointers to Objects:**

Pointers to objects are useful for creating objects at run time. To access members arrow operator ( ) and de referencing operator or indirection (\*) are used.

### **Declaration of pointer:**

```
className*ptr
```

ex:

```
item *obj;
```

Here obj is a pointer to object of type item.

```
class item
```

```
{
```

```
    int code;
```

```
    float price;
```

```
public:
```

```
    void getdata(int a,float b)
```

```
    {
```

```
        code=a;
```

```
        price=b;
```

```
    }
```

```
    void show()
```

```
    {
```

```
        cout<<"code:"<<code<<"\n"<<"Price:"<<price<<endl;
```

```
    };
```

### **Declaration of object and pointer to class item:**

```
item obj;
```

```
item *ptr=&obj;
```

The member can be accessed as follow.

#### **a) Accessing members using dot**

```
operator obj.getdata(101,77.7);
```

```
obj.show();
```

#### **b)using pointer**

```
Ptr-> getdata(101,77.7);
```

```
ptr->show();
```

#### **c)Using de referencing operator and dot operator**

```
(*ptr).getdata(101,77.7);
```

```
(*ptr).show();
```

### **Creating array of objects using pointer:**

```
item *ptr=new item[10];
```

Above declaration creates memory space for an array of 10 objects of type item.

```
#include<iostream.h>
```

```
class item
```

```
{
```

```
int code;
```

```
float price;
```

```
public:
```

```
void getdata(int a,float b)
```

```

{
code=a;
price=b;
}

void show()

{
cout<<code<<"\t"<<price<<endl;
}

};

int main()
{
int n;
int cd;
float pri;
cout<<"Enter number of objects to be created:";
cin>>n;
item *ptr=new item[n];
item *p;
p=ptr;
for(int i=0;i<n;i++)
{
cout<<"Enter data for object"<<i+1;
cout<<"\nEnter Code:";cin>>cd;
cout<<"Enter price:";cin>>pri;

```

```

p->getdata(cd,pri);

p++;

}

p=ptr;

cout<<"Data in various objects are "<<endl;

cout<<"Sno\tCode\tPrice\n";

for(i=0;i<n;i++)

{

cout<<i+1<<"\t";

ptr->show();

ptr++;

}

return 0;

}

```

### **Pointers to Derived Classes:**

Pointers can be declared to derived class. It can be used to access members of base class and derived class. A base class pointer can also be used to point to object of derived class but it can access only members that are inherited from base class.

```
#include<iostream.h>
```

```
class base
```

```
{ public:
```

```
int a;
```

```
void get_a(int x)
```

```
{
```

```

a=x;

}

void display_a()
{
    cout<<"In base"<<"\n"<<"a="<<a<<endl;
}

};

class derived:public base
{
int b;

public:
void get_ab(int x,int y)
{
a=x;
b=y;
}

void display_ab()
{cout<<"In Derived "<<"\n"<<"a="<<a<<"\nb="<<b<<endl;
}

};

int main()
{
base b;

base *bptr;

```

```

bptr=&b;           //points to the object of base class
bptr->get_a(100);
bptr->display_a();
derived d;
derived *dptr;
dptr=&d;           //points to the object of derived class
dptr->get_a(400);
dptr->display_a();
dptr->get_ab(300,200);
dptr->display_ab();
bptr=&d;           //points to the object of derived class
bptr->get_a(400);
bptr->display_a();
return 0;
}

```

### **Output:**

#### **In base**

a=100

#### **In base**

a=400

#### **In Derived**

a=300

b=200

#### **In base**

$$a=400$$